

《计算机系统基础实践》任务书

实验一 数据的表示和等效运算

一、实验目的与要求

(1) 熟练掌握程序开发平台(VS2019/VS2022/VS2023) 的基本用法, 包括程序的编译、链接和调试;

(2) 熟悉数据的表示形式;

(3) 熟悉地址的计算方法、地址的内存转换;

(4) 用常见的按位操作(移位、按位与/或/非/异或) 等实现运算表达式的等效运算。

二、实验内容

任务1 数据存放的压缩与解压编程

定义了 结构 student , 以及结构数组变量 beforecompress[N], decompress[N]; (N=5)

```
struct student {  
    char  name[8];  
    short age;  
    float score;  
    char  remark[200]; // 备注信息  
};
```

编写程序, 将 beforecompress[N] 中的所有信息依次紧凑(压缩)存放到一个字符数组 message 中, 然后从 message 解压缩到结构数组 decompress[N]中。

要求:

(1) 输入的第 0 个人姓名(name)为自己的名字, 分数为学号的最后两位;

(2) 编写指定接口的函数完成数据压缩

压缩函数有两个: `int pack_student_bytebybyte(student* s, int sno, char *buf);`

`int pack_student_whole(student* s, int sno, char *buf);`

s 为待压缩数组的起始地址; sno 为压缩人数; buf 为压缩存储区的首地址; 两个函数的返回均是调用函数压缩后的字节数。pack_student_bytebybyte 要求一个字节一个字节的向 buf 中写数据; pack_student_whole 要求对 short、float 字段都只能用一条语句整体写入, 用 strcpy 实现串的写入。

(3) 使用指定方式调用压缩函数

beforecompress 数组的前 N1 (N1=2) 个记录压缩调用 pack_student_bytebybyte 完成; 后 N2 (N2=3) 个记录压缩调用 pack_student_whole, 两种压缩函数都只调用 1 次。

(4) 使用指定的函数完成数据的解压

解压函数的格式: `int restore_student(char *buf, int len, student* s);`

buf 为压缩区域存储区的首地址; len 为 buf 中存放数据的长度; s 为存放解压数据的结构数组的起始地址; 返回解压的人数。解压时不允许使用函数接口之外的信息(即不允许定义其他全局变量)

(5) 仿照调试时看到的内存数据, 以十六进制的形式, 输出 message 的前 40 个字节的

内容，并与调试时在内存窗口观察到的 message 的前 40 个字节比较是否一致。

(6) 打印压缩前(beforecompress)、解压后(decompress)的结果，以及压缩前、压缩后存放数据的长度。

(7) 对于第 0 个学生的 score，根据浮点数的编码规则指出其个部分的编码，并与观察到的内存表示比较，验证是否一致。

(8) 指出结构数组中个元素的存放规律，指出字符串数组、short 类型的数、float 型的数的存放规律。

任务 2 编写位运算程序

按照要求完成给定的功能，并**自动判断程序**的运行结果是否正确。（从逻辑电路与门、或门、非门等等角度，实现 CPU 的常见功能。所谓自动判断，即用简单的方式实现指定功能，并判断两个函数的输出是否相同。）

(1) int absVal(int x); 返回 x 的绝对值

仅使用 !、~、&、^、|、+、<<、>>，运算次数不超过 10 次

判断函数：int absVal_standard(int x) { return (x < 0) ? -x : x; }

(2) int negate(int x); 不使用负号，实现 -x

判断函数：int netgate_standard(int x) { return -x; }

(3) int bitAnd(int x, int y); 仅使用 ~ 和 |，实现 &

判断函数：int bitAnd_standard(int x, int y) { return x & y; }

(4) int bitOr(int x, int y); 仅使用 ~ 和 &，实现 |

(5) int bitXor(int x, int y); 仅使用 ~ 和 &，实现 ^

(6) int isTmax(int x); 判断 x 是否为最大的正整数 (7FFFFFFF)，

只能使用 !、~、&、^、|、+

(7) int bitCount(int x); 统计 x 的二进制表示中 1 的个数

只能使用，!~&^|+<<>>，运算次数不超过 40 次

(8) int bitMask(int highbit, int lowbit); 产生从 lowbit 到 highbit 全为 1，其他位为 0 的数。例如 bitMask(5,3)=0x38；要求只使用 !~&^|+<<>>；运算次数不超过 16 次。

(9) int addOK(int x, int y); 当 x+y 会产生溢出时返回 1，否则返回 0

仅使用 !、~、&、^、|、+、<<、>>，运算次数不超过 20 次

(10) int byteSwap(int x, int n, int m); 将 x 的第 n 个字节与第 m 个字节交换，返回交换后的结果。n、m 的取值在 0~3 之间。

例：byteSwap(0x12345678, 1, 3) = 0x56341278

byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD

仅使用 !、~、&、^、|、+、<<、>>，运算次数不超过 25 次

(11) int bang(int x) 当 x=0 时，返回 1；其他情况返回 0。实现逻辑非(!)

例：bang(3)=0; bang(0)=1;

仅使用 ~、&、^、|、+、<<、>>，运算次数不超过 12 次

提示：只有当 x=0 时，x 与 -x 的最高二进制位会同时为 0。

(12) int bitParity(int x) 当 x 有奇数个二进制位 0，返回 1；否则返回 0

例：bitParity(5)=0; bitParity(7)=1;

仅使用 `!`、`~`、`&`、`^`、`|`、`+`、`<<`、`>>`，运算次数不超过 20 次。

提示：只有当 `x` 的高字与低字的对应位（第 0 位对应第 16 位，第 1 位对应第 17 位，依次类推）同时为 1，则出现了成对的二进制位 1，此时，可以将对应的二进制位置为 0，不会影响二进制位 1 的个数的奇偶性判断。